Technische Hochschule Rosenheim

Prüfungsstudienarbeit (PStA)

Speech-based Detection of Alzheimer's Disease

The PARLO Dementia Corpus

What this PStA is (and is not) about

- **Goal:** you understand a **field** and show it by approaching a real problem **scientifically**
- **Not the goal:** the most elaborate code or the highest score
- What we look at in your work:
  - Do you understand the **clinical problem** and the **data**?
  - Do you formulate **questions / hypotheses** and justify your design choices?
  - Do you **evaluate** honestly and **interpret** the results (including negative ones)?
  - Do you know the **limitations** of your approach and data?
- Code is a tool to answer your questions, you should understand what you did and explain it

Structure of the PStA

- exercises prep for certain concepts, you can use in the PStA
- every team must check-in regularly with the lecturer (progress, questions, problems)
- **PStA report** (4 pages, paper style, PDF) + runnable **code**
- **Presentation** (15 min + 5 min Q&A) at the end of the semester

Dementia and Alzheimer's Disease (AD)

- **Dementia** = umbrella term:
  - acquired decline in cognition (memory, language, executive function, ...) that **interferes with everyday life**
- **Alzheimer's disease** is the most common cause (the majority of cases); others: vascular, Lewy body, frontotemporal dementia
- Progressive and, so far, **not curable**; new disease-modifying drugs work best **early**
- Burden grows with ageing societies: tens of millions affected worldwide (WHO), strong impact on patients, families and health systems

Typical course

1. Memory problems (new information, recent events)
2. Word-finding, language and orientation problems
3. Loss of independence in daily activities
4. Severe impairment, full dependence on care

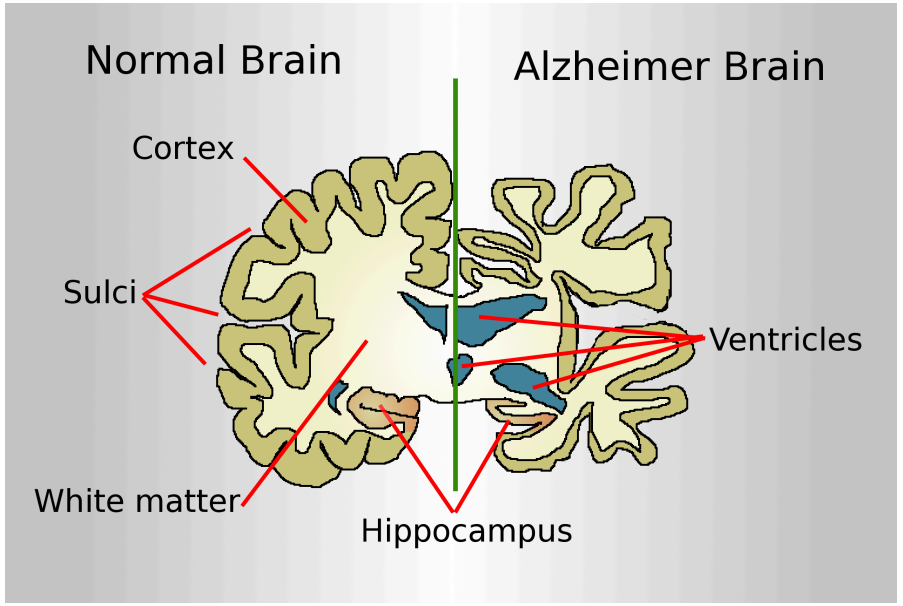
What happens in the brain?

- **β-Amyloid plaques** and **tau tangles** accumulate; neurons and synapses are lost (atrophy, starting in the medial temporal lobe / hippocampus)
- Pathology begins **years to decades** before symptoms
- Today's **biological definition** (NIA-AA research framework) is based on biomarkers:
  - CSF: β-Amyloid (↓), p-Tau (↑)
  - Imaging: MRI (atrophy), PET (amyloid / tau / metabolism)

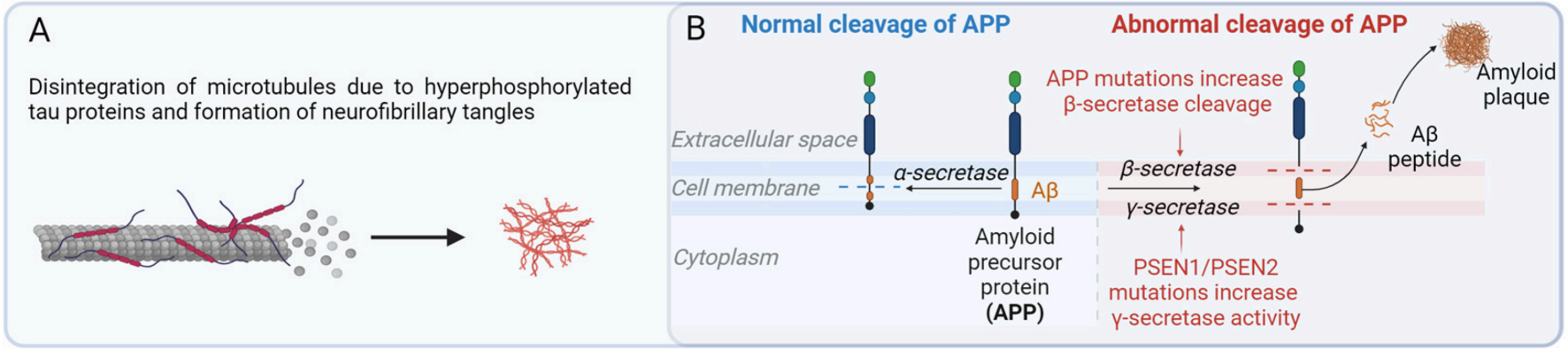
Disease continuum

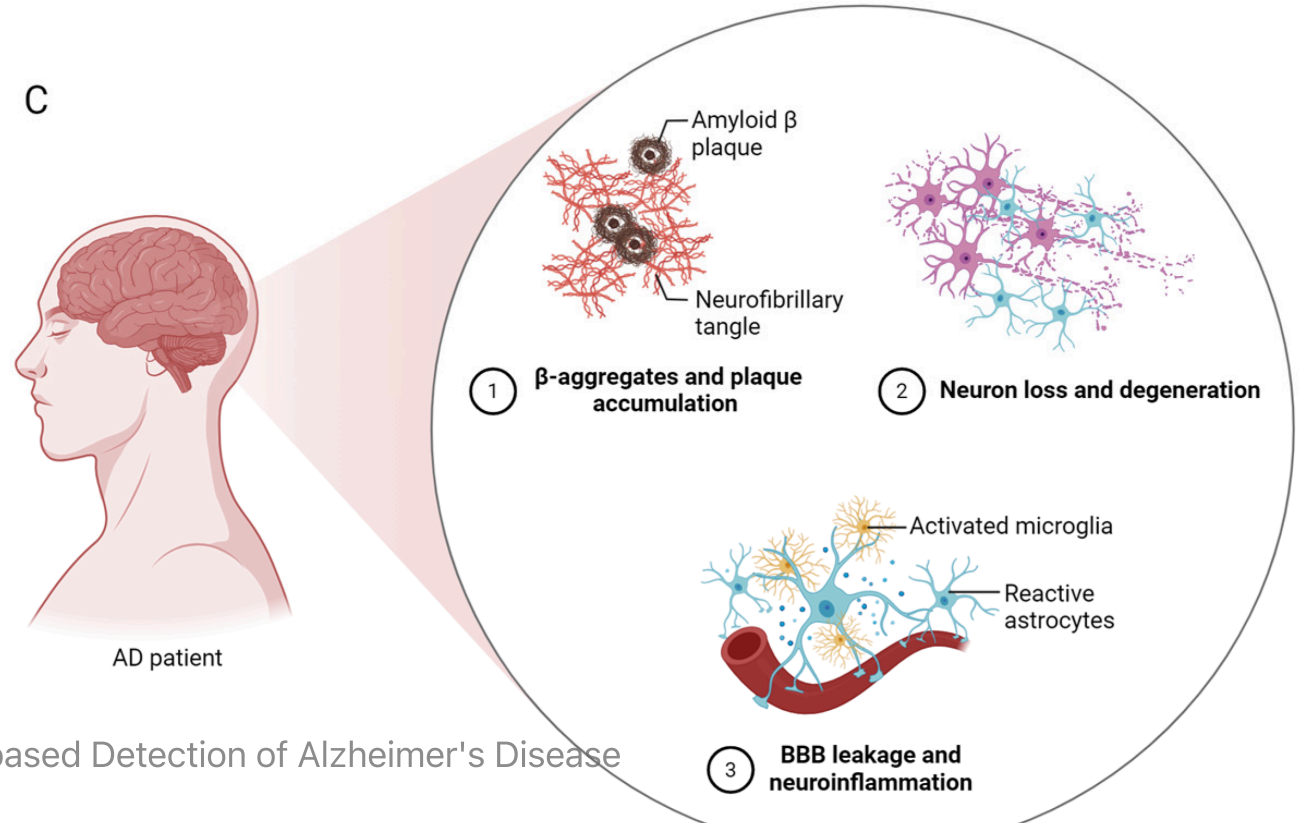
`cognitively healthy (HC)` → `mild cognitive impairment (MCI)` → `mild → moderate → severe dementia (DEM)`

- Boundaries are **gradual**, labels are clinical conventions
- This continuum is exactly what your classifiers have to cut into classes



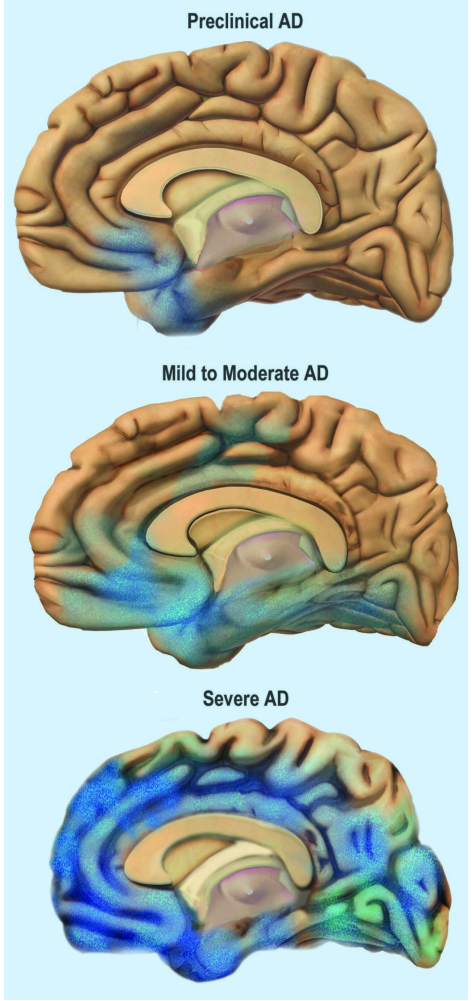
Neuropathological features of AD





Alzheimer's Disease Spreads through the Brain

- symptoms appear **after** pathology has spread to the neocortex
- early stages: **hippocampus** (memory), **temporal lobe** (language, semantic memory)
- later stages: **frontal lobe** (executive function, planning), **parietal lobe** (visuospatial function)



Mild Cognitive Impairment (MCI)

- Cognitive decline **greater than expected for age and education**, but **daily life is (largely) preserved**
- Often the stage where intervention would be most valuable
- **Heterogeneous:** MCI has many causes, only some are due to AD ("AD-related MCI"); some people remain stable or even recover
- Many MCI patients convert to dementia over the years (rates differ strongly between cohorts)

Why this matters for you

- MCI is the **hardest class**: subtle differences, overlapping with HC and DEM
- Expect **confusions between neighboring classes** (see baseline results later)
- Think about whether a 3-class or a binary problem answers *your* question

How is it diagnosed today?

Instrument	What it is
MMSE (Mini-Mental State Examination)	30-point screening test; lower = worse; coarse, ceiling effects in early stages
CERAD neuropsychological battery	detailed test battery with age-adjusted norms
CDR (Clinical Dementia Rating)	clinician rating of overall severity (0 = none, 0.5 = questionable, 1 / 2 / 3 = mild / moderate / severe)
CSF biomarkers	β-Amyloid, p-Tau via lumbar puncture (invasive)
Imaging	MRI, PET (expensive, not available everywhere)

- Biomarkers: **costly and invasive**; cognitive tests: **subjective in scoring**, need trained staff, practice effects
- Need: **cheap, scalable, objective, repeatable** screening

Why speech?

Cognition → Speech

Speaking is a **highly demanding cognitive task**:

1. **Conceptualization** (what to say) - semantic memory, executive control
2. **Lexical retrieval** (which word) - lexical access
3. **Syntax / planning** - working memory
4. **Articulation** - speech-motor control

AD affects the first three much earlier than articulation.

Practical advantages

- non-invasive, cheap
- remote, repeated, unobtrusive (phone, tablet)
- tasks can be standardized
- rich signal: **what** is said (text) and **how** it is said (audio)

Caveat

Speech is also affected by age, education, mood, dialect, hearing, language, device, ...

What changes in the speech of people with AD / MCI?

Acoustic / temporal

- more and longer **pauses**, slower speech rate
- more hesitations / fillers ("ähm")
- changes in prosody (variability of pitch, rhythm)
- reduced voice quality in later stages

Linguistic

- **word-finding difficulties**, circumlocutions, generic words ("thing", "this")
- reduced **vocabulary / semantic fluency**
- less **information content** in picture descriptions
- repetitions, repairs, simpler syntax
- worse **recall** of stories / pictures (episodic memory)

- Findings come from small cohorts, often one language and one task (see systematic reviews later)

Related Work and Existing Datasets

- **DementiaBank / Pitt Corpus** (Becker et al., 1994): English, "Cookie Theft" picture description; most widely used
- **ADReSS / ADReSSo** (Interspeech 2020 / 2021), **TAUKADIAL** (2024): challenges with age/gender-balanced, preprocessed data
- Limitations: small, English-centric, mostly **one task**, rarely standardized tests, German almost absent

Two families of approaches

1. **Feature pipelines:** hand-crafted acoustic + linguistic features → classical classifier (SVM, RF, gradient boosting)
2. **Representation learning:** (fine-tune) pre-trained audio / text encoders (wav2vec, BERT, …); often fused (multi-modal)

- Reviews point out: heterogeneous methods, **limited external validation**, dataset biases, hard to compare

The PARLO Dementia Corpus (PDC) at a Glance

Braun, Witzl, Hönig, Nöth, Bocklet, Riedhammer (2026): *The PARLO Dementia Corpus: A German Multi-Center Resource for Alzheimer's Disease*, LREC 2026

- First **German, multi-center, clinically validated** corpus for AD-related MCI and dementia
- **9 academic memory clinics** in Germany, study by the PARLO Institute
- **208 speakers**, ≈ 20 h of audio, 8 speech tasks per person (not everyone completed all)
- Manually verified **transcriptions** + demographic, clinical and **biomarker metadata**

Group	n	Age (mean ± sd)	Gender
Healthy controls (HC)	83	55-87 (69.0 ± 7.9)	32 m / 51 f
MCI (AD-related)	59	55-85 (70.7 ± 8.4)	32 m / 27 f
Dementia (DEM)	66	55-85 (71.7 ± 8.8)	32 m / 34 f
All	**208**	55-87 (70.4 ± 8.4)	96 m / 112 f

- Small (!) and slightly different gender mix per group - think about what that means for modeling

Who is in the Data? (Inclusion / Exclusion)

- **Patients:** established AD diagnosis according to **NIA-AA criteria**; MCI to moderate dementia
- **Controls:** CERAD scores within norms (≤ 1.2 sd below age-adjusted norms), ≥ 6 years of education, no subjective cognitive complaints
- All: ≥ 55 years, fluent German, adequate vision / hearing, informed consent
- **Excluded** e.g.: depression, Parkinson's, brain tumor / major head trauma, cerebrovascular disease, psychoactive medication interfering with testing

Consequences for generalization

- A **clean, selected** population: real screening populations are messier (comorbidities, depression, other dementias)
- Results on PDC do **not automatically** transfer to the clinic

The PARLO Dementia Test Battery (8 tasks, tablet-based)

#	Task	What the speaker does	Cognitive domain
1	Story Reading	reads a short story aloud	baseline for prosody, fluency, articulation
2	Confrontation Naming	names 15 line drawings (Boston Naming variant)	lexical access, semantics
3	Animal Naming	names as many animals as possible in 1 min	semantic fluency, executive control
4	Picture Description	describes a "Mountain Scene"	spontaneous discourse, planning
5	Repetition "pataka"	repeats the pseudo-word for 10 s	phonology, speech-motor planning
6	Repetition "sischafu"	same with a second pseudo-word	phonology, less item effects
7	Story Recall	retells the story after distraction	**episodic memory**, narrative
8	Picture Recall	describes the picture from memory	**episodic memory**, spatial / semantic

- Tasks range from **constrained** (naming) to **open-ended** (recall): different cognitive load per task
- Which task is informative for **which** question? That is one of your design decisions

Recordings, Transcripts and Metadata

Audio

- same **iPad** model and app at all sites, quiet room, 16-bit mono, 44.1 kHz; clips 4 s to 4.5 min
- may contain short **examiner interventions**

Transcripts

- manual, verbatim, Dresing & Pehl rules; dialect → standard German
- annotated: **pauses** `(.) (..) (3)`, **fillers** (ähm), overlaps, laughter, unintelligible parts, speaker changes

Metadata

- age, gender, **education**, site / clinic ID, subject ID, session info
- diagnosis group, **MMSE**, CDR, CSF biomarkers (β-Amyloid, p-Tau), imaging

Baseline 1: ASR on Disordered Speech

- Three open systems: **Whisper large-v3**, NVIDIA **Parakeet-tdt-0.6b-v3**, **OWSM-CTC v4**; evaluated with **WER** against manual transcripts

Whisper WER (%)	HC	MCI	DEM
Story Reading	5.0	8.2	15.2
Picture Description	7.6	10.5	18.5
Story Recall	10.8	17.7	25.2
Animal Naming	13.9	21.6	18.9

- WER **increases with impairment** (mostly) and with **less constrained tasks**; Whisper is best overall
- Look closely: Animal Naming does **not** follow the trend (DEM < MCI), and in some cells Parakeet beats Whisper - **read tables critically**
- Take-away: **ASR errors are themselves group-dependent** and propagate into any text-based pipeline

Baseline 2 and 3: Automatic Scoring and LLM Classification

Automatic test scoring (Confrontation Naming, Animal Naming)

- rule-based: ASR transcript → normalize → match against word lists (GermaNet, OpenThesaurus, Levenshtein ≥ 85 %) → count
- Pearson r, ASR-based vs. manual score: **0.92** (Confrontation Naming), **0.99** (Animal Naming)
- vs. MMSE: 0.43 / 0.65; sum of both: **0.70**

Zero-shot LLM classification (HC vs. MCI vs. DEM)

- Mistral-3.1 (24B), prompts in German, Whisper transcripts, metric **UAR** (unweighted average recall)
- Stimulus task only: UAR **42.7 %** (story) / **45.6 %** (picture)
- **Adding the recall transcript: 59.3 % / 66.0 %** - biggest gain for **MCI**
- Recall (memory-driven speech) carries diagnostic information; chance level for 3 classes is 33 %

Evaluation Pitfalls You Must Think About

The paper recommends **speaker-independent cross-validation** and **leave-one-site-out** validation. Why?

- **Speaker leakage:** several recordings per person; the same speaker in train and test = optimistic scores
- **Site effects:** 9 clinics - model may learn site / examiner instead of disease
- **Confounders:** age, gender, education differ between groups or correlate with the label
- **Small data (208 speakers)** and **class imbalance** → high variance; use **UAR / macro-F1**, report spread, not only one number
- **Tuning on the test set** (hyperparameters, feature selection, task choice)
- **Pipeline errors:** ASR quality differs by group; transcripts (manual vs. ASR) in training and test must match your deployment assumption
- **Labels are clinical conventions**, MCI is heterogeneous; MMSE is a coarse proxy

Approaching the Problem Scientifically

Before you code, write down:

1. **Question / hypothesis:** e.g. "Which task or modality is most informative for separating MCI from HC?", "Does the recall task add information over the stimulus task?"
2. **Task definition:** classes (3-class, binary, regression on MMSE), which subset, which tasks
3. **Representation:** acoustic, linguistic, or both - and **why** (link to the clinical literature)
4. **Baseline & ablation:** what is the simplest reasonable baseline; which component do you remove / add to test your hypothesis?
5. **Evaluation protocol:** splits, metric, uncertainty, comparison to the paper's baselines
6. **Error analysis:** which classes, which speakers, which tasks fail - and what do you conclude clinically?
7. **Limitations & ethics:** what does your result *not* tell us? Would you trust it in a clinic?

Required and Recommended Reading

Required

Required

- **PDC paper:** Braun et al. (2026). *The PARLO Dementia Corpus*. LREC 2026. https://lrec.elra.info/lrec2026-main-753 (arXiv:2603.03471)
- **NIA-AA framework:** Jack et al. (2018). *Alzheimer's & Dementia*, 14(4):535-562

Required and Recommended Reading

Reviews (speech and AD)

- de la Fuente Garcia, Ritchie, Luz (2020). *J. Alzheimer's Disease*, 78(4):1547-1574
- Martínez-Nicolás et al. (2021). *Frontiers in Psychology*, 12
- Vigo, Coelho, Reis (2022). *Bioengineering*, 9(1)

Datasets and challenges

- Becker et al. (1994), Pitt Corpus; Luz et al. (2020, 2021) ADReSS / ADReSSo; Barrera-Altuna et al. (2024) TAUKADIAL

Methods

- Balagopalan et al. (2020), *To BERT or not to BERT*, Interspeech
- Braun et al. (2022 / 2023 / 2024): earlier PDC-based studies (Interspeech, TSD)
- Radford et al. (2023), Whisper, ICML

PStA Requirements

Mandatory

- Three class classification using speech and text features (acoustic, linguistic, or both)
  - minimum: 2 x unimodal baselines, + fusion model
  - recreate the method from one paper (of your choice, agreed upon with me) working with Audio + Text features for (dementia) classification

Optional

- ASR pipeline (optional, but recommended)
- Picture description prototype (web app/ App...)

Deliverables:

- i. 4-page paper describing your method and results (PDF)
- ii. runnable code (access to repo)
- iii. presentation (15 min + 5 min Q&A)

PStA Requirements

- coding agents are explicitly allowed, but you must understand and explain what they did
- you are expected to **understand the clinical problem, the data, and the methods** you use
- Small internal challenge on performance, best UAR on the 3-class classification (HC, MCI, DEM) wins

Legal requirements:

- You must **not** share the data outside the course (e.g., on GitHub, Colab, etc.)
- this includes uploading the data to
  - cloud services (e.g., Google Drive, Dropbox, OneDrive, iCloud)
  - ASR services (e.g., Whisper API, Google Speech-to-Text, AWS Transcribe)
  - LLM providers (e.g., ChatGPT, LLaMA, Claude, etc.)

PStA Timeline / Milestones (WS 26/27)

Date	Milestone
22.10.26	Team registration, Data available
29.10.26	"Kick-Off" Workshop
12.11.26	Check-In 1, literature, what do you want to build?
03.12.26	Check-In 2, progress meeting
10.12.26	preliminary results and preliminary paper due, discussion of ideas for the remainder of the semester, own ida/ literature
17.12.26	intermediate presentation (preliminary paper feedback)
24.12.26	Christmas Break
31.12.26	Christmas Break
07.01.27	Check-In 3 (remove final obstacles)
19.01.27	Final Paper due
21.01.27	Presentation, Final Paper due

Next: Machine Learning Projects and Pipelines

- CRISP-DM / ML project workflow
- Evaluation: confusion matrix, UAR, F1, cross-validation, hyperparameter tuning
- Pipelines in scikit-learn

Evaluation of (Supervised) Learning Models

AI outperformed average radiologists with an accuracy of 82.9%

source: Najar Najafi N, Hajihassani H, Azimzadeh Irani M. The Impact of Artificial Intelligence on Cancer Diagnosis and Treatment: A Review. Cancer Inform. 2025 Sep

Classification

- **Classification** is a type of supervised learning where the goal is to predict a discrete label for a given input based on training data.
- e.g. decide if an image contains a Dog, Cat, Horse, Unicorn, etc.
- so you often hear, a classification model is 99% accurate, but what does that mean?

What kind of classification outcomes are there?

Accuracy

- **Accuracy** is on of the most common metric used to evaluate classification models as it easy to understand and intuitive

$$\text{Accuracy} = \frac{\text{correctly classified instances}}{\text{total instances}} = \frac{TP + TN}{TP + FP + FN + TN}$$

Let's compare two classifiers!

- imagine you're hunting a fairy tale creature, the legendary unicorn
- you have two classifiers that predict if your camera just discovered a unicorn or if it's just another common horse
- which one do you choose?

Motivation

Evaluation of the models shown to you:

- **Classifier 1**: Correctly classified **80.00%** of the 1000 samples it was shown correctly
- **Classifier 2**: Correctly classified **99.9%** of the 1000 samples it was shown correctly

- **It's a no-brainer, right? Classifier 2 is the better choice!**

- **or is it?**

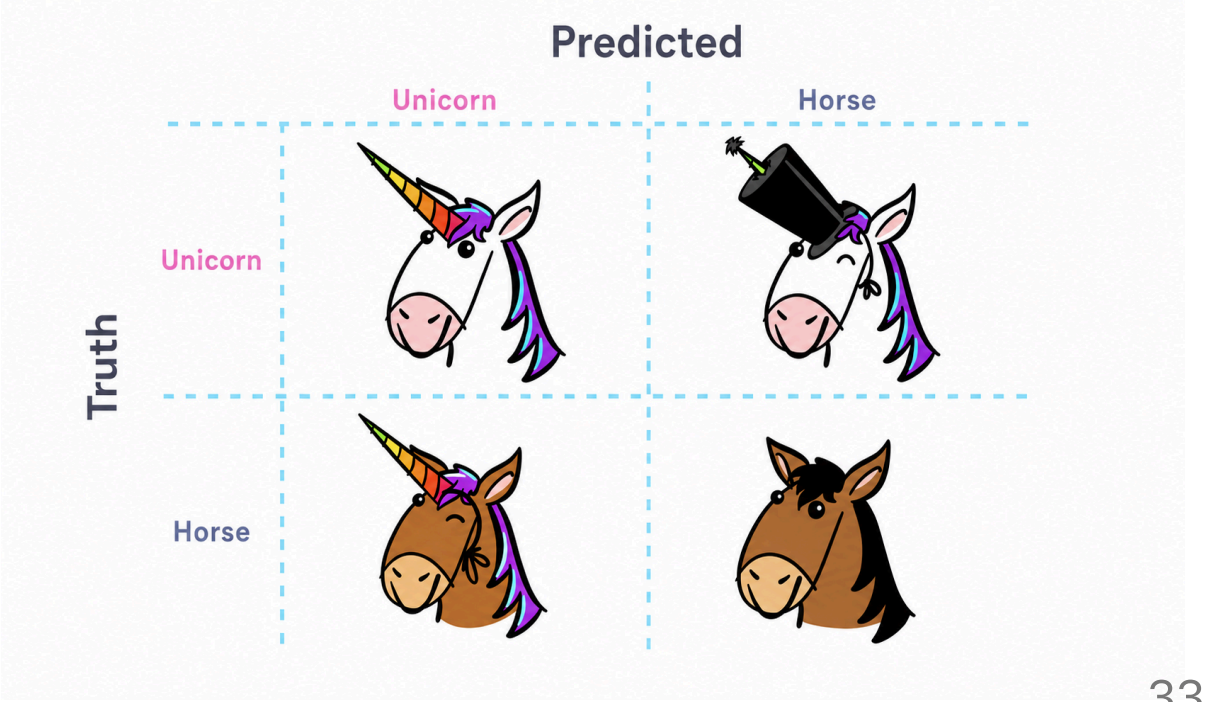
Confusion Matrix

A **confusion matrix** is a Table visualizing the performance of a classifier
allows describing the performance of a classification model on a set of test data for which the true values are known.

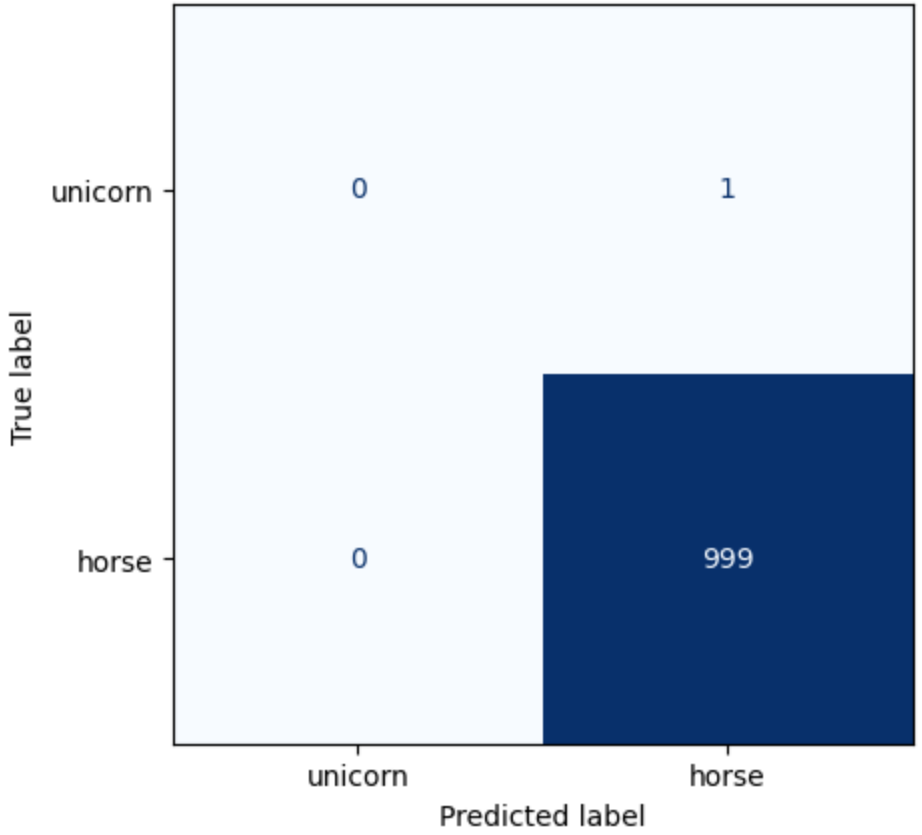
- TP **(True Positives)**: The number of instances correctly classified as the positive class.
- FP **(False Positives)**: The number of instances incorrectly classified as the positive class.
- FN **(False Negatives)**: The number of instances incorrectly classified as the negative class.
- TN **(True Negatives)**: The number of instances correctly classified as the negative class.

Formally, for a binary classification problem, a confusion matrix M is a 2x2 matrix defined as:

$$M = \begin{pmatrix} TP & FN \\ FP & TN \end{pmatrix}$$



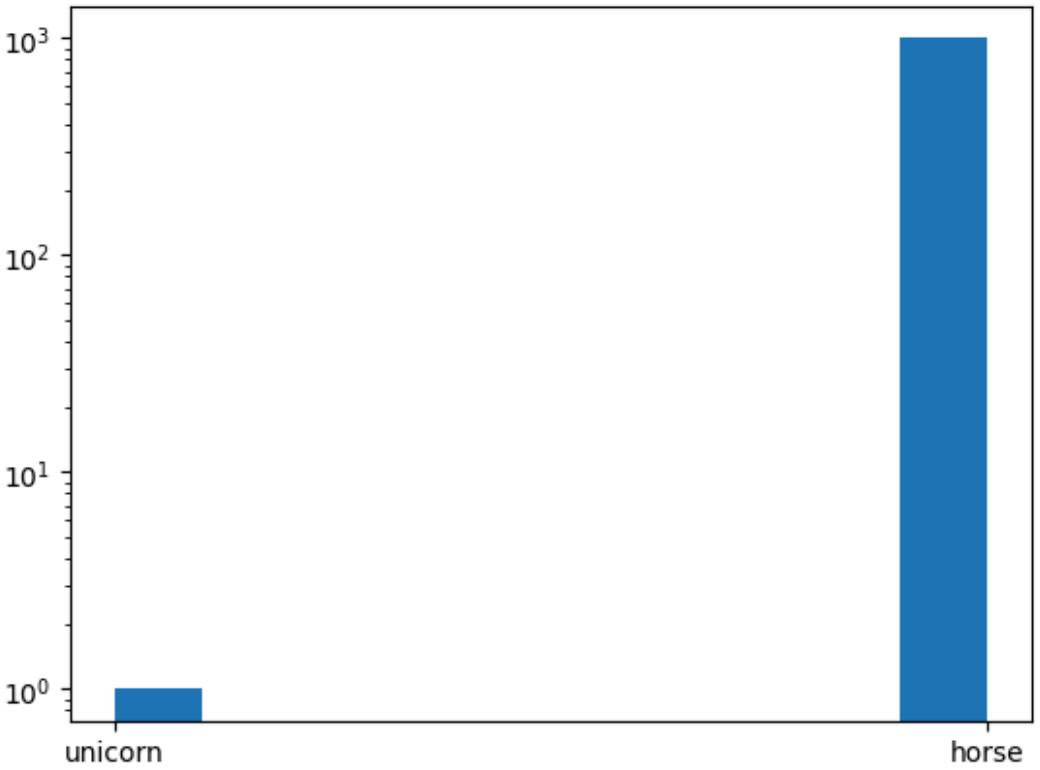
Confusion Matrix



Back to our unicorn example:

Looking at the confusion matrix, we can see that **Classifier 2** doesnt have any True Positives, which means it never found the unicorn.

Motivation



- Turns out, the unicorn is a rare creature, who would have thought?
- **Classifier 2** has a **99.9%** accuracy, but it never found the unicorn when it was shown
- it missed the one thing you were looking for
- in unbalanced datasets, accuracy is not always the best metric to evaluate a model
- it might mask the fact that the model is not able to detect the rare class at all

Accuracy Confusion Matrix Example

- **Accuracy** is on of the most common metric used to evaluate classification models as it easy to understand and intuitive

$$\text{Accuracy} = \frac{\text{correctly classified instances}}{\text{total instances}} = \frac{TP + TN}{TP + FP + FN + TN}$$

Example

	pos	neg
pos	10	15
neg	20	100

- $$\frac{10 + 100}{10 + 15 + 20 + 100} = 75.86\%$$

	pos	neg
pos	0	25
neg	0	120

- $$\frac{120}{10 + 15 + 20 + 100} = 82.75\%$$

Evaluation of Classification Models with Confusion Matrix based metrics

✎ Confusion Matrix based evaluation metrics

- Accuracy
- Precision
- Recall
- F1-Score
- Sensitivity and Specificity

Reflection

So what would you say to someone who told you, that we don't need radiologists anymore, because AI (with 82.9% accuracy) is better at detecting cancer than humans?

Recall

- Recall determines "how complete" the decisions of the model are regarding a specific class
- Defined as the fraction of correctly classified positive instances (of one class) out of all actual positive instances (of that class)

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall is the answer to the question: What percentage of the positive instances of a class did the model correctly classify?

	pos	neg
pos	10	15
neg	20	100

- $r_0 = \frac{10}{10 + 15} = 40.0\%$
- $r_1 = \frac{100}{20 + 100} = 83.3\%$

Precision

- **Precision** determines how precise decisions in favor of a class k are, i.e., the fraction of correct assignments to class k among all assignments to class k:

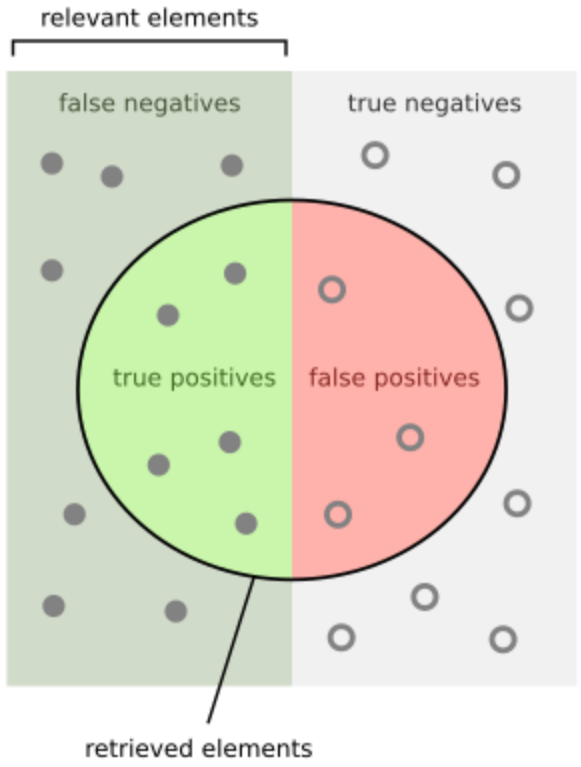
$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision is the answer to the question: What percentage of the instances classified as positive by the model are actually positive?

	pos	neg
pos	10	15
neg	20	100

- $p_0 = \frac{10}{10 + 20} = 33.3\%$
- $p_1 = \frac{100}{15 + 100} = 86.9\%$

Precision and Recall, TP, TN, FP, FN visualized



How many retrieved items are relevant?

Precision =

How many relevant items are retrieved?

Recall =

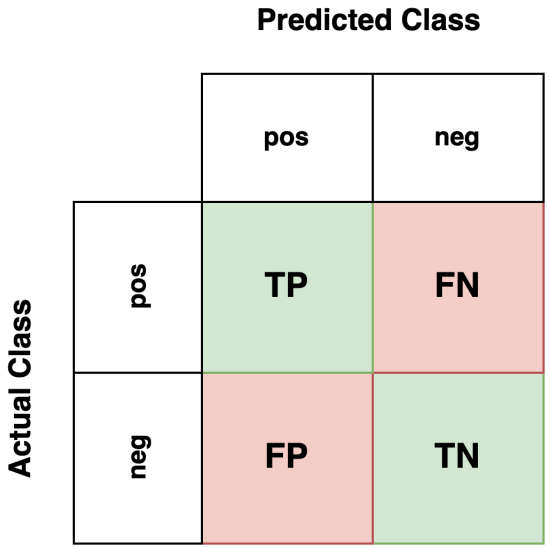
Sensitivity and Specificity

Sensitivity is the same as Recall, but is often used in the context of medical diagnostics
Specificity is **not** the same as Precision. It is also often used in the context of medical diagnostics

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Specificity is the answer to the question:
What percentage of actual negative instances were correctly classified as negative?

- **"recall for the negative class"**



F1-Score

- harmonic mean of precision and recall
- F1-Score is a metric that combines both precision and recall

$$F_1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

- Why the harmonic mean?
- The harmonic mean is used to calculate the average of rates, e.g., speed, ratios
- the F1-Score is high only when both precision and recall are high
- penalizes extreme values more than the arithmetic mean
- F1 is close to 1 only if both precision and recall are high
- F1 is close to 0 if either precision or recall is very low
- it is possible to modify the F1-Score to give more weight to precision or recall (F_β-Score)

Multi-Class Classification Metrics Example

Average Precision, Recall, F1-Score

- **Macro Average**: Calculate the metric for each class and then average the results
  - treats all classes equally
  - also called **unweighted average**
- **Micro Average**: aggregates the counts (true positives, false negatives, and false positives) over all classes and then calculates the overall metric.
  - treats all instances equally
  - this is a form of **weighted average**
- **Weighted Average**: Calculate the metric for each class and then average the results, weighted by the number of instances in each class
  - treats all classes by their prevalence
  - you want their contribution to reflect how often they appear in real data.

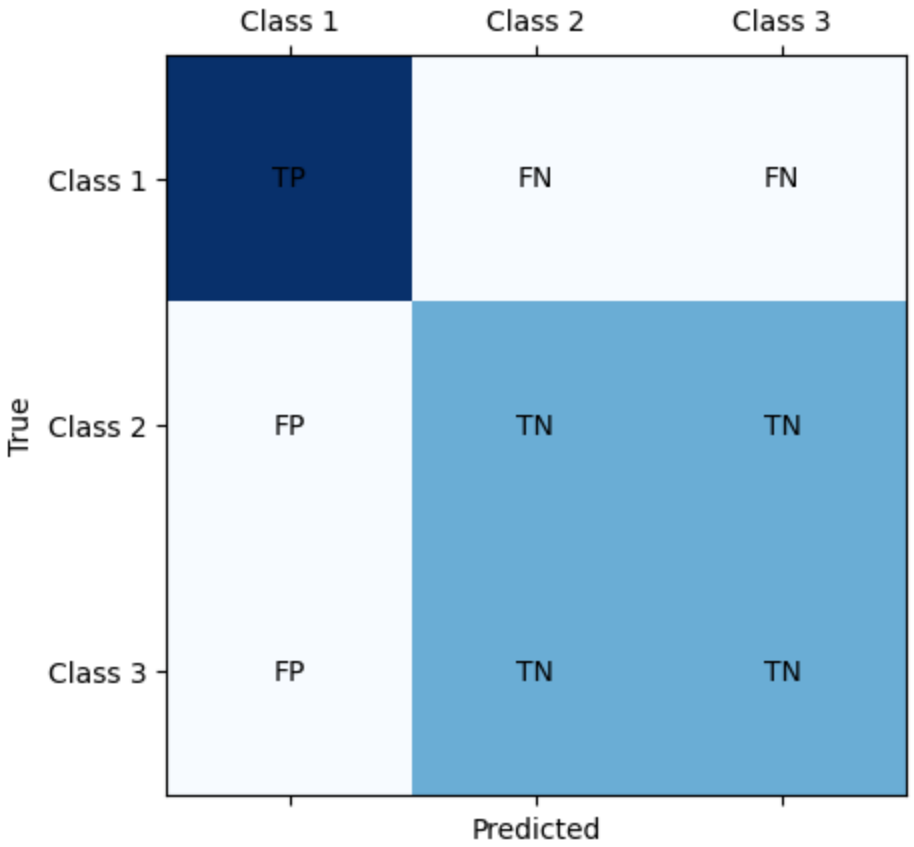
Averaging (only what we'll use in this course)

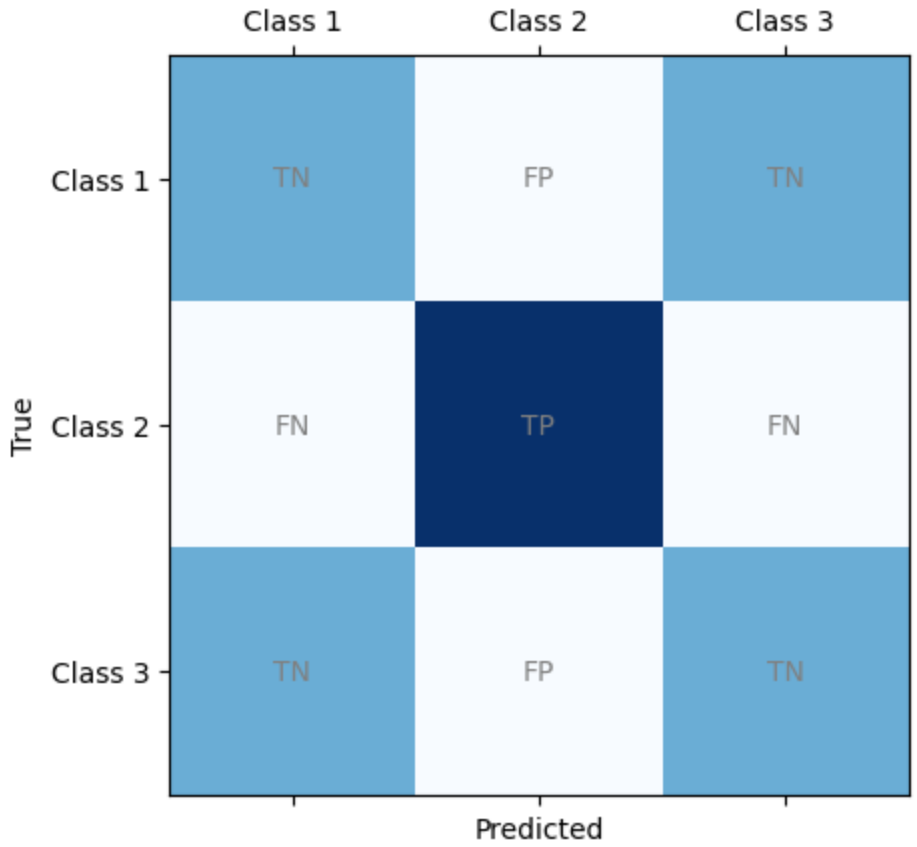
1. Macro Averaging

- **Mathematical Definition**: Macro averaging calculates the metric for each class separately and then takes the simple (unweighted) average of these metrics. With C as the number of classes:
- $\text{Macro Precision} = \frac{1}{C} \sum_{i=1}^{C} \text{Precision}_i$
- $\text{Macro Recall} = \frac{1}{C} \sum_{i=1}^{C} \text{Recall}_i$

Multi-class confusion matrix

- TP, TN, FP, FN are defined for each class





Check your understanding 1

- Which averaging method would you use if you want to give equal importance to all classes, regardless of their frequency in the dataset?
- Which averaging method would you use if you want to reflect the performance o, all classes are equally important. speech emo recognition the model on the most frequent classes more heavily?
- In a highly imbalanced dataset, how would the micro and weighted averages differ in their representation of model performance?

Check your understanding 2

You run a company that provides an API for detecting fraudulent transactions.
The fraud detection model has been shown to be 98.8% accurate on a test set.
Your model has the following confusion matrix on a test set:

	Predicted Fraud	Predicted Legitimate
Actual Fraud	50	10
Actual Legitimate	200	17400

More information: Your company makes $0.1 per transaction it processes. You lose $100 per fraudulent transaction that goes undetected. You lose $50 per legitimate transaction that is flagged as fraud. Based on these metrics, discuss whether your model is suitable for deployment in a real-world scenario?

Check your understanding 3

How likely do you have the Influenza given that you have a positive test result?

- Prevalence of Influenza in the population: 1%
- Test sensitivity (recall): 99%
- Test specificity: 95%

Sidenote:

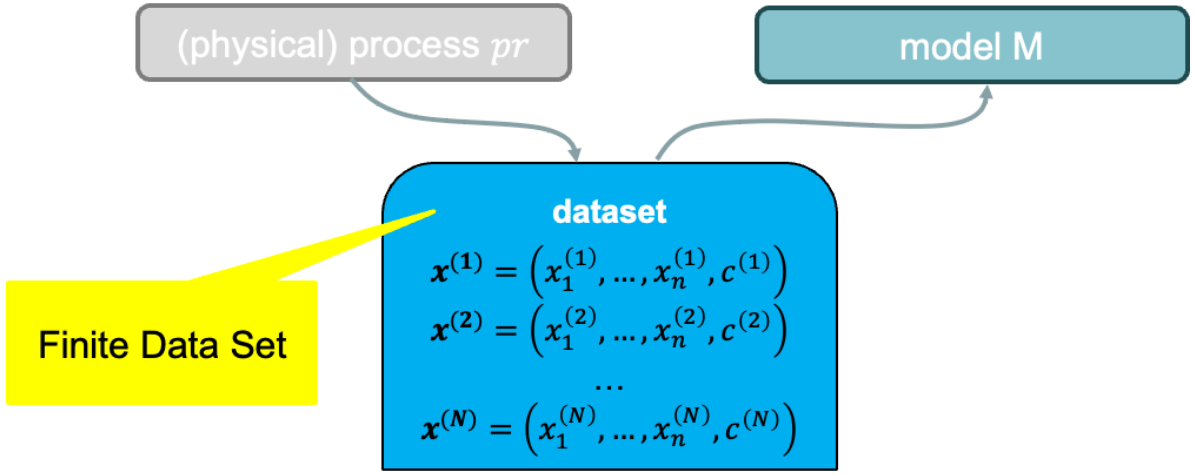
- minimum requirements for COVID-19 tests: 97% specificity, 80% sensitivity
- why so low? under the assumption, that only people with symptoms were tested
- Blanket testing would lead to many false positives, as the prevalence is low

Estimating Performance

Challenge:

1. Which is the best classification/ regression algorithm for a given dataset?
2. Which is the most suitable parametrization of an algorithm for a given task?

- our dataset is limited, it is a random sample from the population
- for evaluation purposes, we calculate $\hat{sc}$, an estimate of sc
- we need to define what a good choice of D is
- $\rightarrow$ sc **is a random variable, we can only estimate it, it's true value is unknown**



Estimating Performance

- We calculate expected the value of the score sc based on a set of observations sampled from the process pr that generates the data

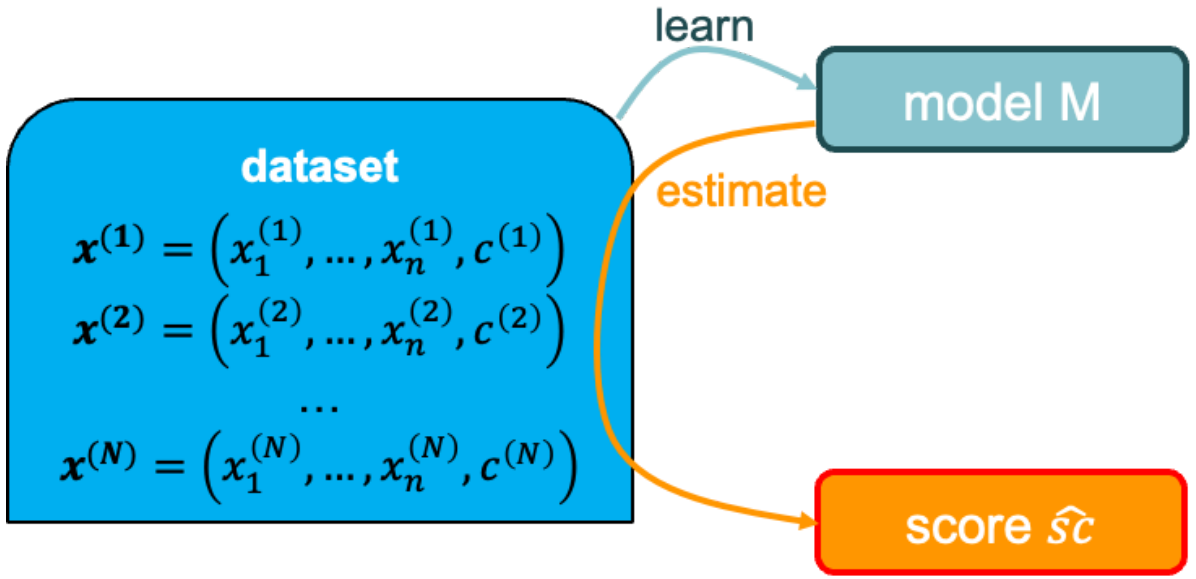
Choices of how to deal with the given dataset:

- Resubstitution
- Hold-Out
- Cross-Validation and Leave-One-Out
- Bootstrap

Resubstitution

- simplest estimation method
- use full dataset D to train the model
- calculate the score sc on the same dataset
- **TOO OPTIMISTIC**, the model has seen the data before, the models will be overfit and the score will quantify the model's ability to memorize the data
- Bad estimator of the true classification score!!!

Every time you test your model on the train set - a puppy dies!

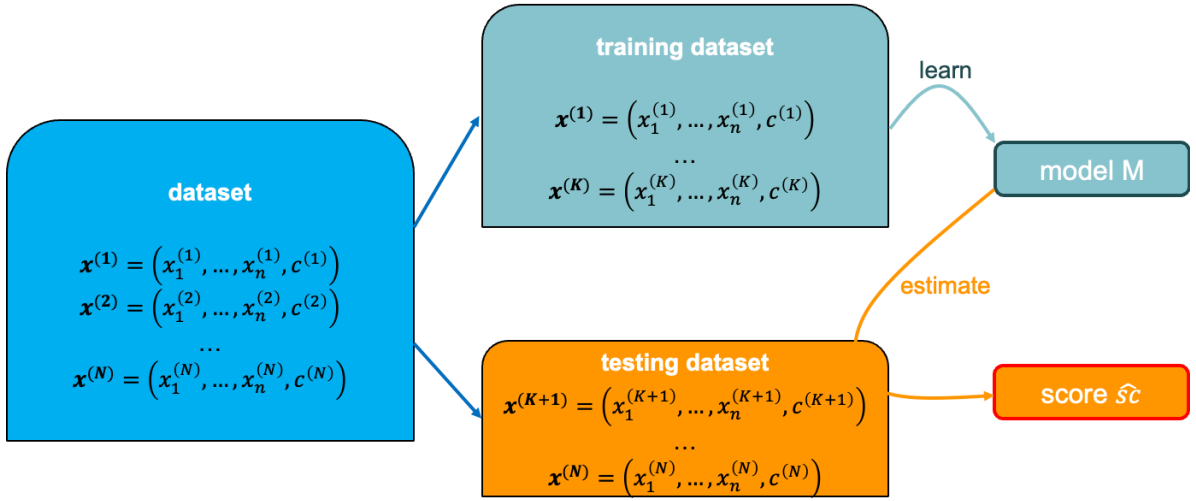


$$D_{tr} = D$$

... and I will be looking at you dissapointetly!

Hold-Out

- split the dataset D into two parts: D_{tr} and D_{te}
- train the model on D_{tr} and calculate the score sc on D_{te}
- **better than resubstitution**, but still not ideal
- can be a pessimistic estimator
- $\rightarrow$ **Useful for large datasets, bad for small datasets**



Hold-Out Discussion

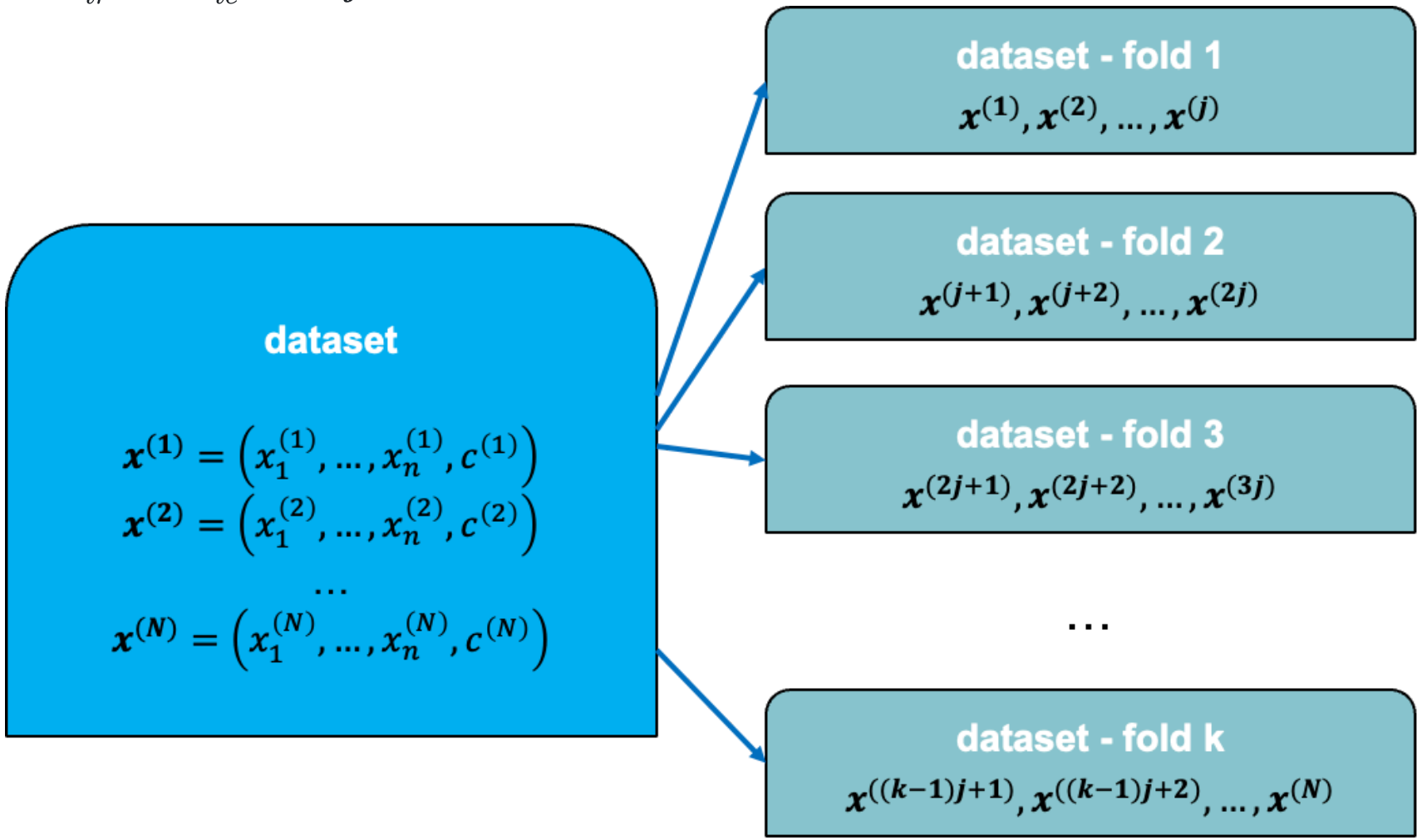
- Better estimate of the score than using full dataset for training and testing
- Can compare score on train and test data to recognize overfitting
  - Good on train, bad on test: overfitting
  - Good on both: model generalizes well
- Issues: validated model only once
  - No way to measure variance of the estimated score
    - The split may by chance have been very conducive to this model
    - The split may have introduced a large skew into the data
  - Significantly reduced size of the training dataset
- → **Useful only for large datasets, but what is “large”?**

You have seen this before, right?:

```
sklearn.model_selection.train_test_split(…)
```

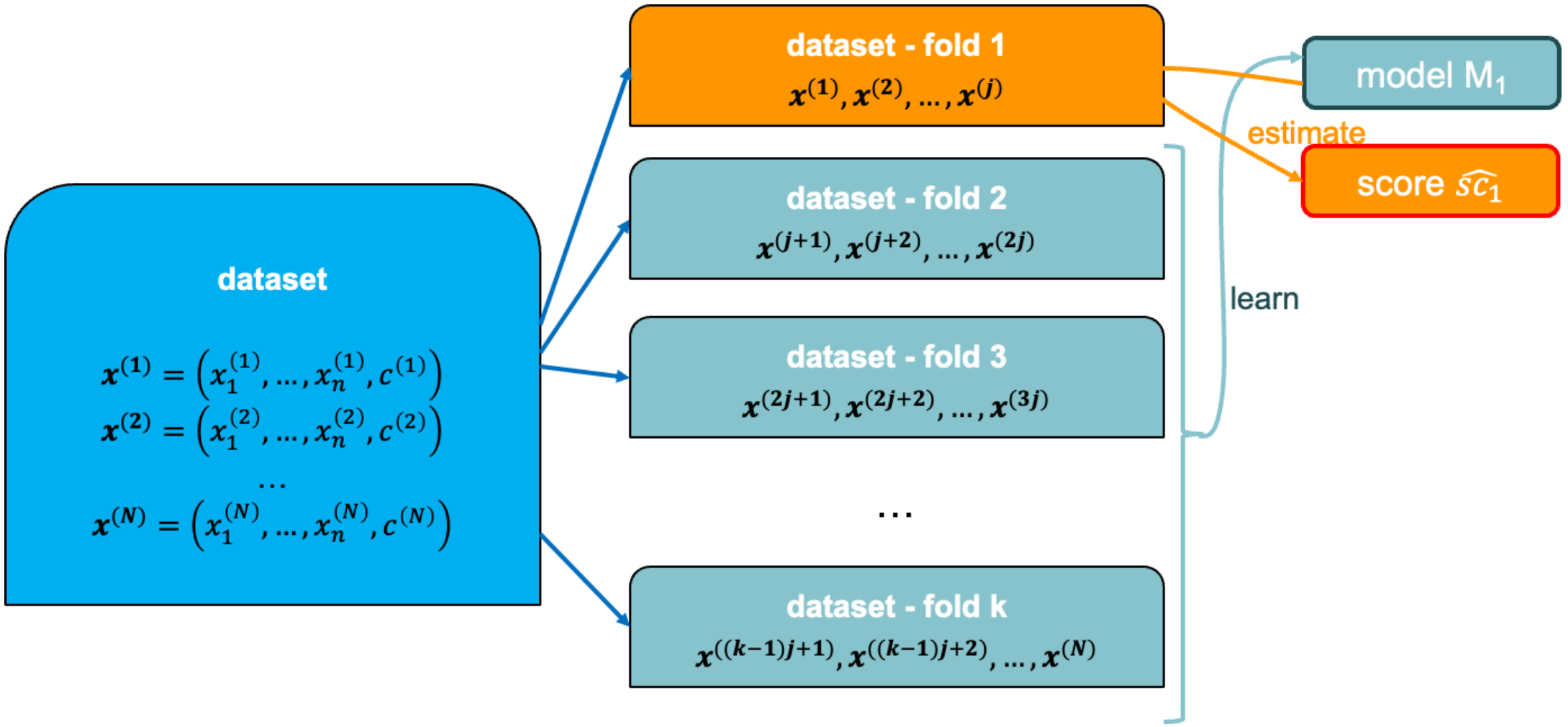
k-fold cross-validation (1)

Let $j = [N/k]$, Define D_{tr}^{j} and D_{te}^{j} as the j-th fold of the training and test set, respectively



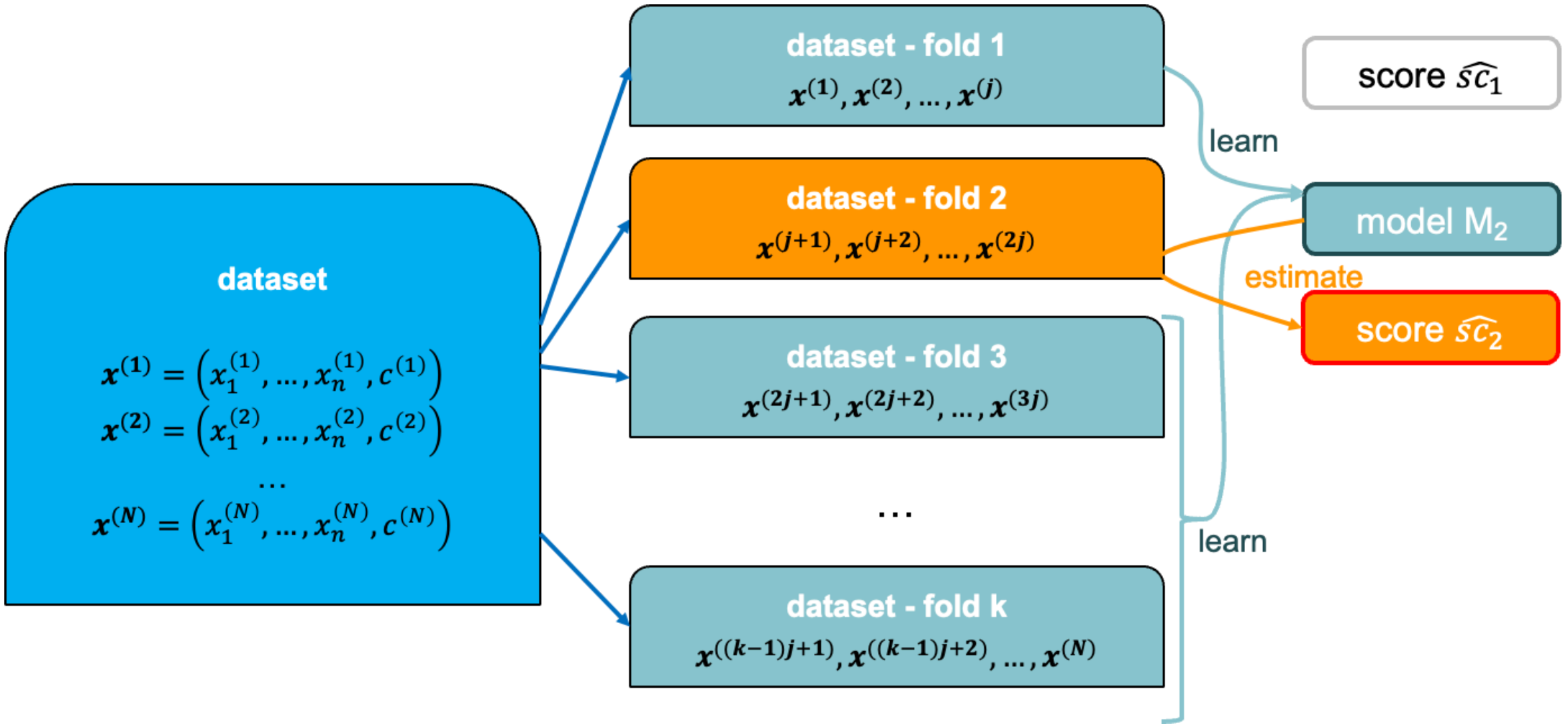
k-fold cross-validation (2)

$UseD_{tr}^{1}$ to train and D_{te}^{1} to calculate the score $\hat{sc}_1$



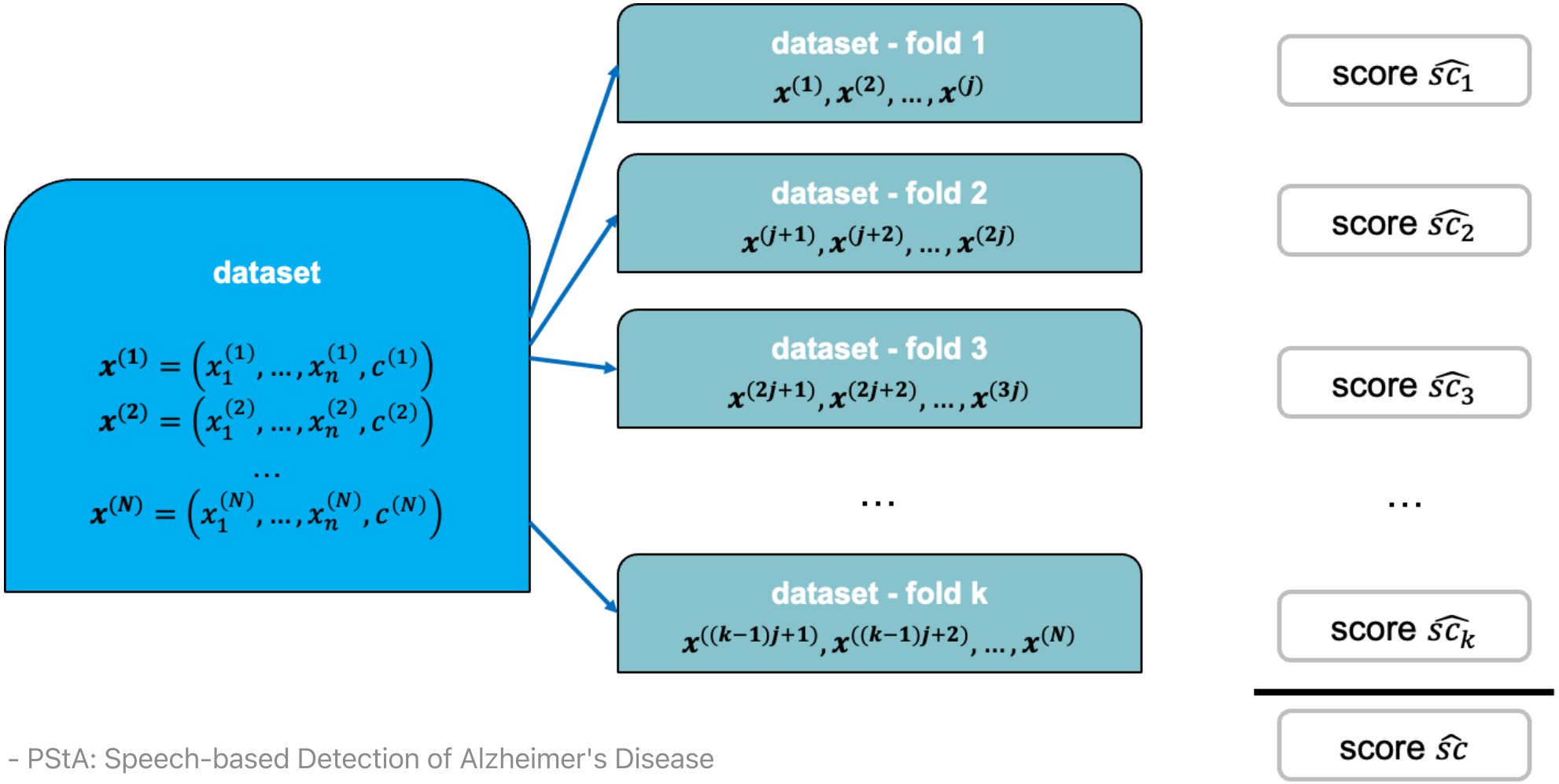
k-fold cross-validation (3)

$Use D_{tr}^1$ to train and D_{te}^1 to calculate the score $\hat{sc}_1$



k-fold cross-validation (4)

- Repeat the process for all k folds
- less pessimistic than hold-out
- → **Suitable for small datasets**



Discussion of k-fold cross-validation

- Validated model k-times
- Reduced size of training data, but less so than for hold-out
- Can compare (mean) score on train and test data to recognize overfitting
  - Good on train, bad on test: overfitting
  - Good on both: model generalizes well
- Can recognize high variance of the model by high variance of the (test) scores
- Challenge: k-times more expensive (training time) than hold-out
  $\rightarrow$ **Superior to Hold-Out**

```
from sklearn.model_selection import KFold
```

Leave-One-Out Cross-Validation

- Special case where $k = N$
- Each fold contains only one observations
- Calculate N scores each based on only one observation (or subject)
- High computational cost
- Better estimation of true score sc, but less stable (=higher variance)

Summary:

Resubstitution

Hold-Out

- Good for large datasets
- Computationally inexpensive
- Frequent splits 80%/20% or 70%/30% (train/test)
- Can be done repeatedly to improve estimation (repeated hold-out)

Cross-Validation and Leave-One-Out

- Good for small datasets
- Computationally expensive

Bootstrap

- .632 Bootstrap good for small datasets
- Computationally expensive

Hyperparameter Tuning

Hyperparameter Tuning

Technische Hochschule Rosenheim

Hyperparameter Tuning

The process of finding the best hyperparameters for a given machine learning model and dataset.

- **Hyperparameters** are set before training and control the behavior of the model
- make or break the performance of the model
- **Hyperparameters** are
  - chosen by the practitioner
  - not learned during training
  - there is no closed-form solution to find the optimal hyperparameters
- found by "trial and error": select a set of hyperparameters, train the model, evaluate the model, repeat

Common methods:

- **Grid Search**: exhaustive search over a specified set of hyperparameter combinations
- **Randomized Search**: randomly selects hyperparameter combinations
- **Bayesian Optimization**: uses a probabilistic model to predict the performance of hyperparameter combinations and selects the most promising ones in each iteration
- ...

Hyperparameter Tuning

Which data to use for evaluation of a set of Hyperparameter Values?

- Cannot use training data – too optimistic
- Cannot use testing data – must not be "tainted" (avoid leakage)
- $\rightarrow$ Need additional dataset, the **validation data** or **development set**

Hyperparameter Tuning and Model Evaluation

- Need to estimate model score to **tune hyperparameters**
  - Pick the optimal hyperparameters based on the score
- Need to estimate model score to **evaluate the model**
  - Recognize Overfitting and evaluate model from the business perspective
- $\rightarrow$ Need to evalute the score twice and independently
- Can chose (any) two different score estimation methods!

Popular Choices

- Hold-Out for both hyperparameter-tuning and model-evaluation: Train-Validation-Test Split (nested-hold-out) – simplest choice
- Cross-validation for hyperparameter-tuning and Hold-Out for model-evaluation
- Cross-validation for both hyperparameter-tuning and model-evaluation: nested cross-validation

Hyperparameter Tuning & Score-Estimation via Train-Validation-Test Split

Split dataset into train, validation and test datasets

- Common splits: 60:20:20 or 50:25:25

Tune Hyperparameters using the validation dataset (repeatedly)

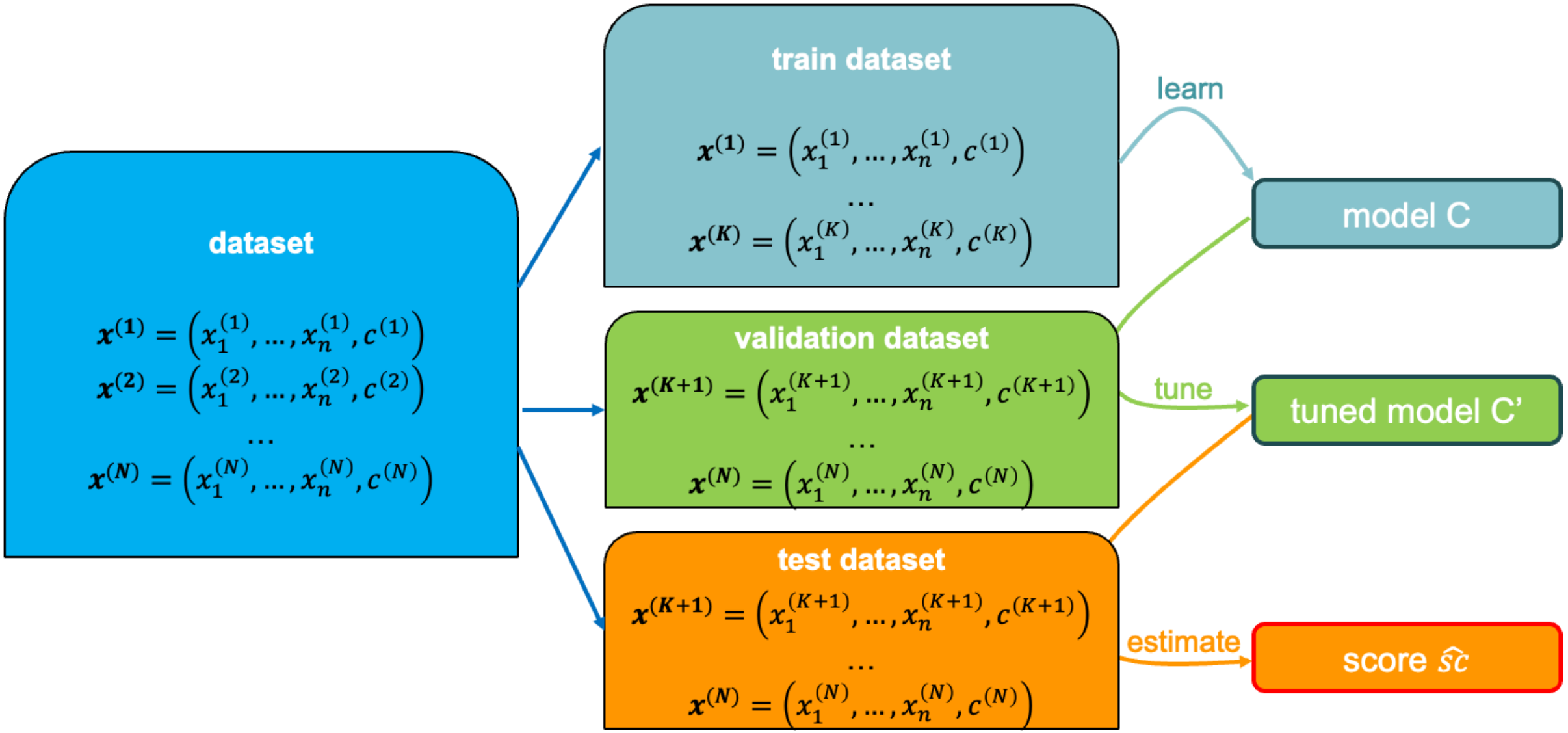
- Train Model on train dataset multiple times for different sets of hyperparameters
- Evaluate score for each set of hyperparameters on validation set
- Pick the best hyperparameters

Compute final performance metric (score) on train+validation and test dataset (once only!)

- Bad on both: underfitting
- Good on train + validation, bad on test dataset: overfitting
- Good on both: model generalizes well

- $\rightarrow$ **Computational cost: one model trained per set of hyperparameter values**

Hyperparameter Tuning & Score-Estimation via Train-Validation-Test Split



Discussion: Train-Validation-Test Split

Advantages

- Test dataset is never used to train/tune – no data leakage
- Good performance
  - model trained once per hyperparameter value set h: $O(h)$
- Easy to implement (simple loop)

Disadvantages – same as (simple) Hold-out (but twice)

- Validated model only once per set of hyperparameter values
  - The split may by chance have been very conducive to this set of hyperparameter values
  - The split may have introduced a large skew into the data
- Validated model only once to evaluate model
  - The split may by chance have been very conducive model
  - The split may have introduced a large skew into the data
- Significantly reduced size of the training dataset

$\rightarrow$ **Useful only for large datasets - but what is "large"?**

```
sklearn.model_selection.train_test_split() # twice
```

Fitting Graph

- Fitting Graph: Plot performance metric over model complexity
  - Example: accuracy over leaf nodes of a Decision Tree
- Used to find the optimal model complexity / to tune hyperparameters visually

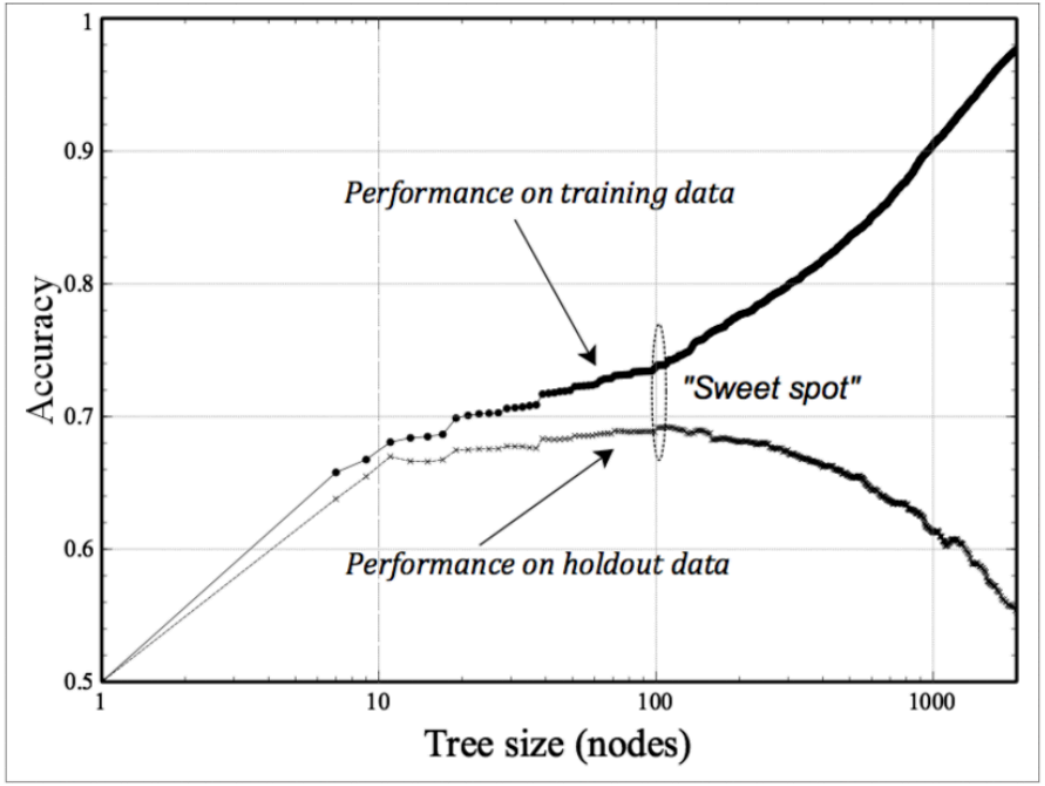


Image source: Foster, Provost, Data Science for Business, p. 117

Cross-validation for hyperparameter-tuning and Hold-Out for model-evaluation

Split dataset into (outer) “train” and test sets and apply k-fold cross-validation to “train”

- Common splits: 80%-20% or 70%-30%

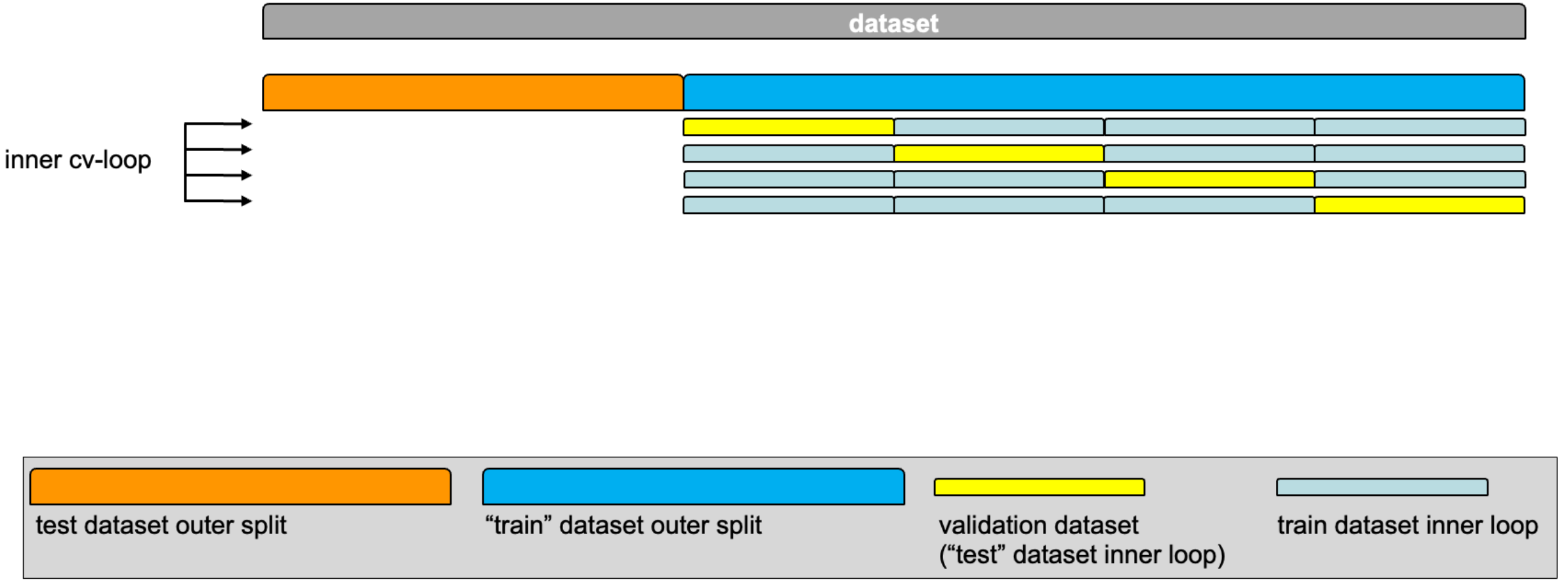
Tune Hyperparameters using cross-validation on the outer “train” data

- For each different set of hyperparameters: train Model k-times (on inner train-folds) and compute score (on inner test-fold). Compute the mean score.
- Pick the best hyperparameters based on the mean score.
- Evaluate variance of the best hyperparameter set

→ Compute final performance metric (score) on (outer) train and (outer) test dataset (once only!)

- Good on train + validation, bad on test dataset: overfitting
- Good on both: model generalizes well

Cross-validation and Hold-Out for Hyperparameter Tuning Visually

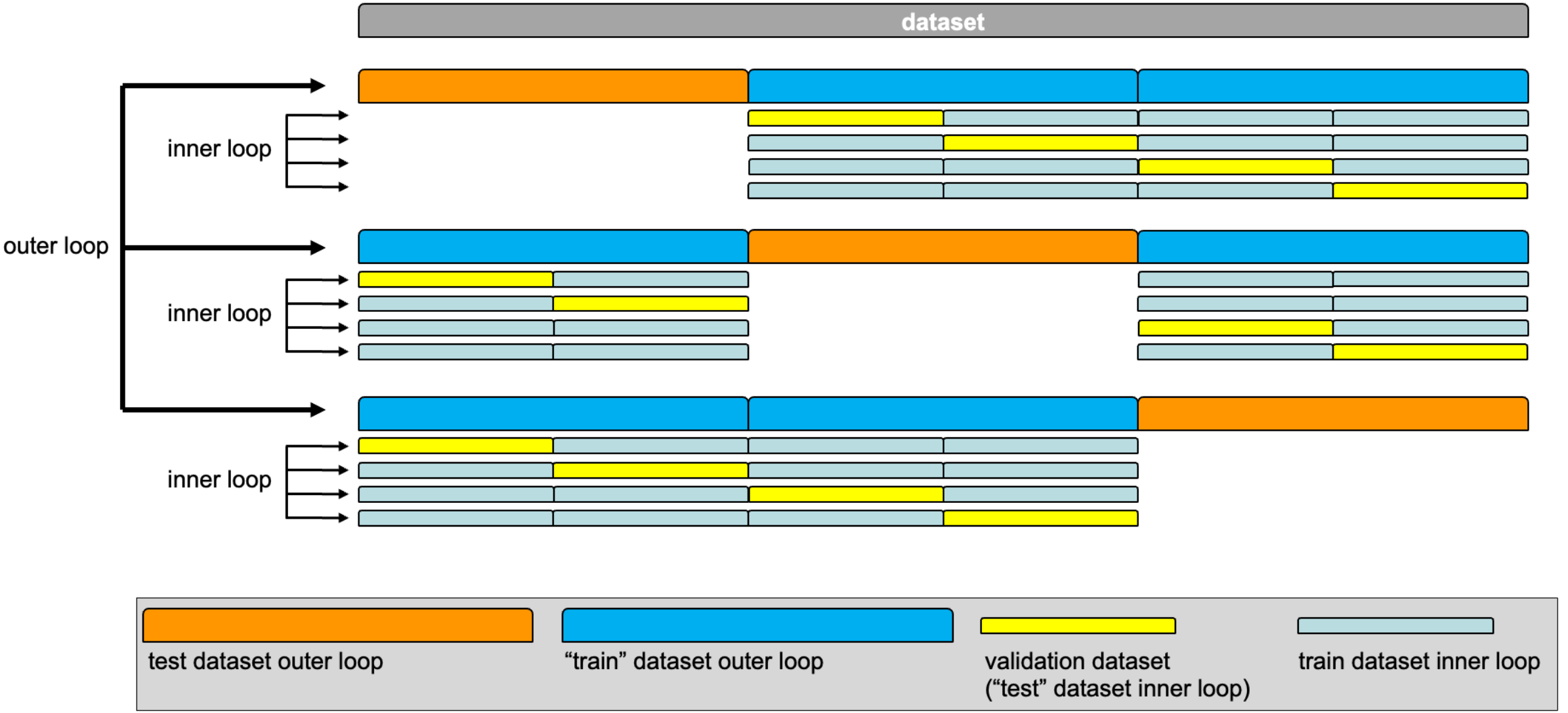


Nested Cross-Validation

Basic Idea: Nest two k-fold-cross validations inside each other

- **Outer Loop** – generates "outer train' and test dataset
  - Facilities model evaluation score as seen before
- **Inner Loop** – generates (real) train and validation from "outer train" dataset
  - Facilities hyperparameter tuning as seen before
- Outer and Inner loop may have different k's ($k_{\text{outer}}, k_{\text{inner}}$)
- **Generalization of Nested-Hold-Out** (nested-hold-out is the special case $k_{\text{outer}} = k_{\text{inner}} = 1$

Nested Cross-Validation – Example for $k_{outer} = 3$ and $k_{inner} = 4$



Training the final model

- Depending on the model evaluation method / the hyperparameter tuning method we may have trained multiple/very many models
- Which one do we deploy to production?

None of them!

- → **We retrain the model (with the optimal hyperparameters we found) on the full dataset to get the final model**
- → **How do we estimate the score of this model?**
  - If we already used cross-validation to estimate the score, we simply use this score
  - If not, we should/may use cross-validation to estimate the score before the final training